****

PPS - UNIT-II

Control Statements:

Selection Statements (decision making) - if and switch statements

Repetition statements (loops)

while, for, do-while statements

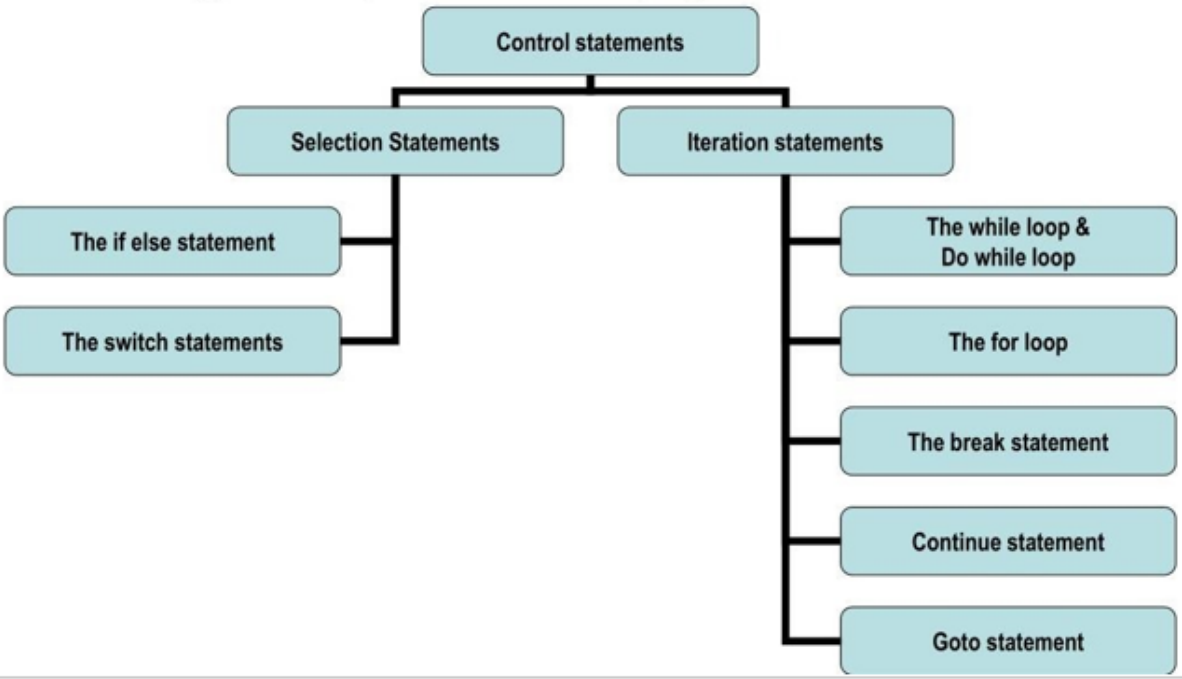
Statements related to looping

break, continue, goto statements

****

Control Structures:

- There may be situations where the programmer requires to alter normal flow of execution of program or to perform the same operation a no. of times.
- Control statements in C are used to write powerful programs by:
  - Repeating important sections of the program
  - Selecting between optional sections of a program



- Various control statements supported by C are: -
  - Decision control statements
  - Loop control statements

Decision control statements/ Selection statements

- Decision control statements alter the normal sequential execution of the statements of the program depending upon the test condition to be carried out at a particular point in program.

- These statements allow us to make decision based upon result of a condition, so that these statements can be called **decision making** or **conditional statements** or sometimes called as **selection statements**, because based on condition it selects the set of instructions to be executed. The following are classified as conditional statements:
- Decision control statements/ Selection statements supported by C are: -
    - **if statement**
    - **switch statement**

if statement:

- The if stamen is powerful decision-making stamen and is used to control the flow of execution of statements.

There are 4 if statements available in C:

1. Simple if statement.

2. if…else statement.

3. Nested if…else statement.

4. else if ladder

1. Simple if statement:

- Simple if statement is used to make a decision based on the available choice. It has the following form:

Syntax: if (condition)

{

stmt block;

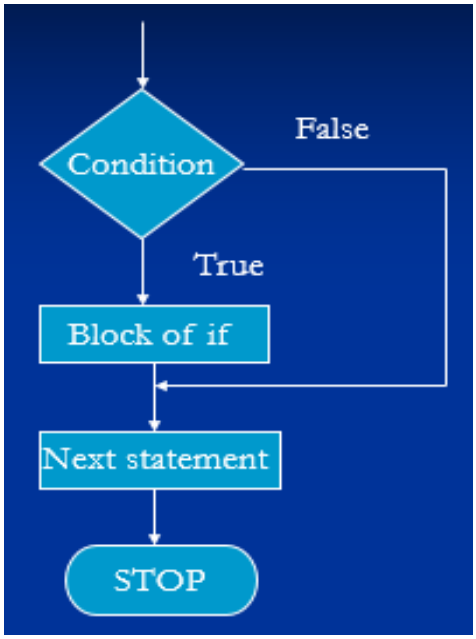
}

stmt-x;

In this syntax,

- if is the keyword. <*condition*>is a relational expression or logical expression or any expression that returns either true or false. It is important to note that the condition should be enclosed within parentheses (_ and _)
- The *stmt block* can be a simple statement or a compound statement or a null statement.
- *stmt-x* is any valid C statement.

The flow of control using simple if statement is determined as follows:



Ex 1: /* Program to check whether a no. is even */

```
# include<stdio.h>
void main() {
int num;
printf("enter the number");
scanf("%d",&num);
if(num%2==0)
{
printf("\n Number is even");
}
printf(" \n End of program"); }
```

OUTPUT:

```
enter the number  6
Number is even
End of program
```

Whenever simple if statement is encountered, first the condition is tested. It returns either true or false. If the condition is false, the control transfers directly to stmt-x without considering the stmt block. If the condition is true, the control enters into the stmt block. Once, the end of stmt block is reached, the control transfers to stmt-x.

Ex 2:

```
#include<stdio.h>
 int main() {
int age;
printf("enter age\n");
scanf("%d",&age);
if(age>=55)
printf("person is retired\n"); }
```

OUTPUT

```
enter age
57
person is retired
```

2. if ….. else statement

- if…else statement is used to make a decision based on two choices.
- It has the following form:

Syntax:

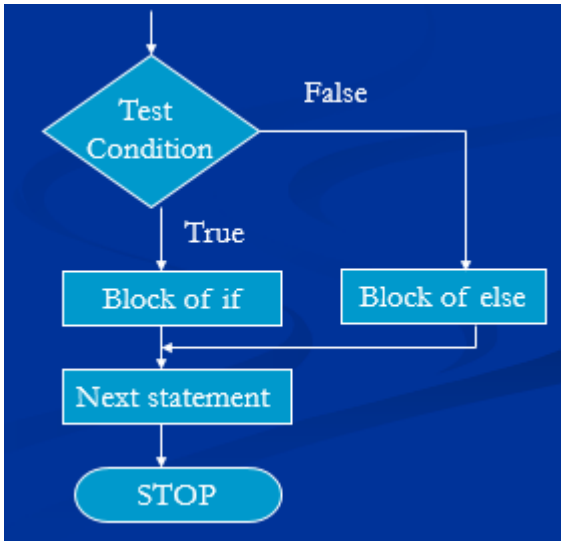
```
if(condition) {
true stmt block;
}
else {
```

false stmt block;
}
stmt-x;

In this syntax,

- if and else are the keywords.
- <*condition*>is a relational expression or logical expression or any expression that returns either true or false. It is important to note that the condition should be enclosed within parentheses (and).
- The *true stmt block* and false stmt block are simple statements or compound statements or null statements.
- *stmt-x* is any valid C statement.

The flow of control using if...else statement is determined as follows:



Whenever if...else statement is encountered, first the condition is tested. It returns either true or false. If the condition is true, the control enters into the true stmt block. Once, the end of true stmt block is reached, the control transfers to stmt-x without considering else-body. If the condition is false, the control enters into the false stmt block by skipping true stmt block. Once, the end of false stmt block is reached, the control transfers to stmt-x.

Program for if else statement:

```
#include<stdio.h>
int main() {
int age;
printf("enter age\n");
scanf("%d",&age);
if(age>=55)
printf("person is retired\n");
else
printf("person is not retired\n"); }
```

OUTPUT

```
enter age
47
person is not retired
```

Ex 2: /* Program to check whether a no. is even or odd */

```
# include<stdio.h>
void main() {
int num;
printf("enter the number");
scanf("%d",&num);
if(num%2==0)
{
printf("\n Number is even");
}
else {
printf("\n Number is odd"); } }
```

OUTPUT

```
enter the number   5
Number is odd
```

3. Nested if—else statement

Nested if…else statement is one of the conditional control-flow statements. If the body of if statement contains at least one if statement, then that if statement is called as ―Nested if…else statement‖. The nested if…else statement can be used in such a situation where at least two conditions should be satisfied in order to execute particular set of instructions. It can also be used to make a decision among multiple choices.

The nested if…else statement has the following form:

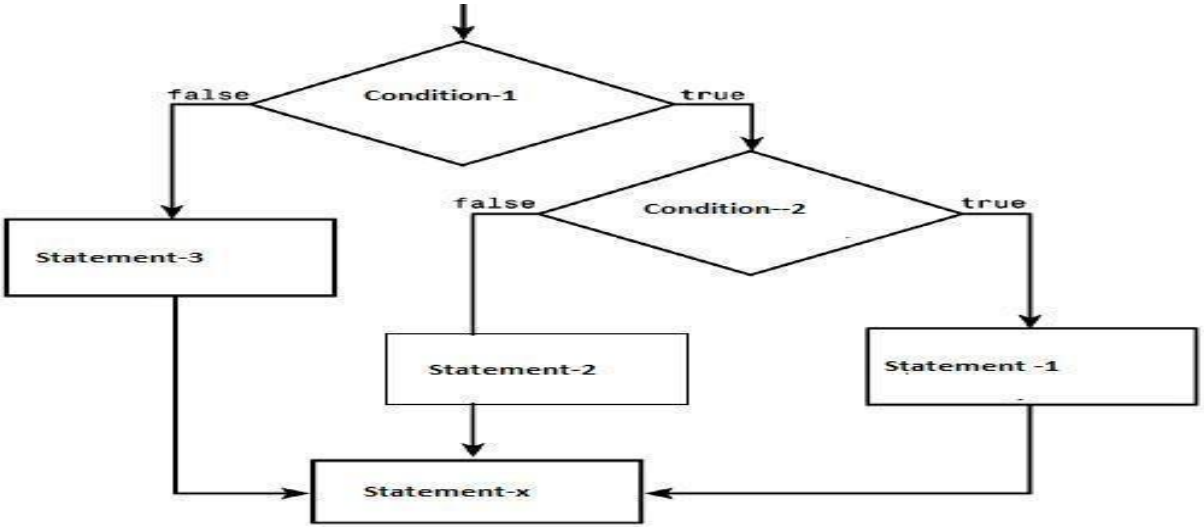
Syntax:

```
if(condition1)
{
if (condition 2) {
stmt1;
}
else {
stmt2;

}

} else {

stmt 3;

}

stmt-x;
```

In this syntax,

- if and else are keywords.

- *<condition1>,<condition2> ... <condition>*are relational expressions or logical expressions or any other expressions that return true or false. It is important to note that the condition should be enclosed within parentheses (and).
- *if-body* and *else-body* are simple statements or compound statements or empty statements.
- *stmt-x* is a valid C statement.



The flow of control using Nested if...else statement is determined as follows:

Whenever nested if…else statement is encountered, first <condition1> is tested. It returns either true or false. If condition1 (or outer condition) is false, then the control transfers to else-body (if exists) by skipping if-body. If condition1 (or outer condition) is true, then condition2 (or inner condition) is tested. If the condition2 is true, if-body gets executed. Otherwise, the else-body that is inside of if statement gets executed.

Program for nested if… else statement:

#include<stdio.h> void main() { int a ,b,c; printf("enter three values\n"); scanf("%d%d%d",&a,&b,&c); if(a>b) { if(a>c) printf("%d\n",a); else printf("%d\n",c); } else {

```
#include<stdio.h>

void main() {

int a ,b,c;

printf("enter three values\n");

scanf("%d%d%d",&a,&b,&c);

if(a>b) {

if(a>c)

printf("%d\n",a);

else

printf("%d\n",c);

} else {
```

```
if(b>c)
printf("%d\n",b);
else
printf("%d\n",b);
}
c;
}
```

OUTPUT

enter three values

3 2 4

4

4. else …… if Ladder

- else-if ladder is one of the conditional control-flow statements. It is used to make a decision among multiple choices. It has the following form:

Syntax:

if(condition 1) {

statement 1;

}

else if(condition 2) {

statement 2;

}

else if(condition 3) {

statement 3;

}

else if(condition n) {

statement n;

}

else {

default statement;

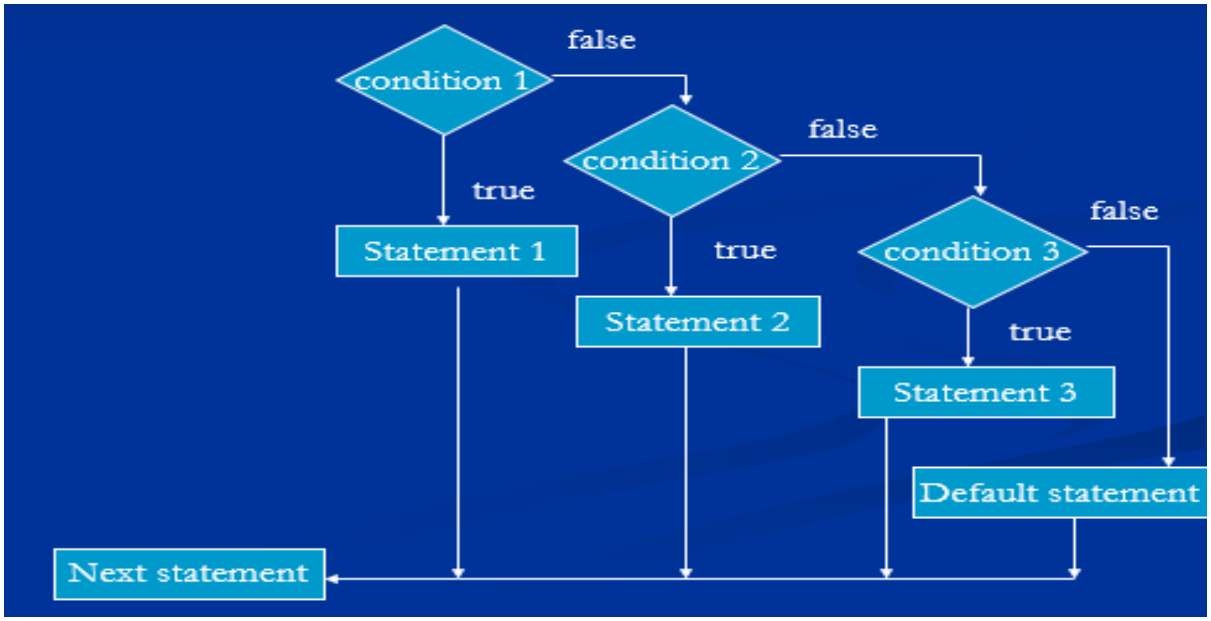
}

stmt- x;

In this syntax,

- if and else are keywords. There should be a space between else and if, if they come together.

- *<condition1>,<condition2>….<condtionN>*are relational expressions or logical expressions or any other expressions that return either true or false. It is important to note that the condition should be enclosed within parentheses (and).
- *statement 1, statement 2,statement 3……,statement n* and *default statement* are either simple statements or compound statements or null statements.
- *stmt-x* is a valid C statement.



The flow of control using else--- if ladder statement is determined as follows:

Whenever else if ladder is encountered, condition1 is tested first. If it is true, the statement 1 gets executed. After then the control transfers to *stmt-x*.

If *condition1* is false, then *condition2* is tested. If *condition2* is false, the other conditions are tested. If all are false, the default stmt at the end gets executed. After then the control transfers to *stmt-x*.

If any one of all conditions is true, then the body associated with it gets executed. After then the control transfers to *stmt-x*.

Example Program:

Program for the else if ladder statement:

```
#include<stdio.h>

void main() {

int m1,m2,m3,avg,tot;

printf("enter three subject marks");

scanf("%d%d%d", &m1,&m2,&m3);

tot=m1+m2+m3; avg=tot/3;

if(avg>=75) {

printf("distinction");
```

```
}
else if(avg>=60 &&avg<75) {
printf("first class");
}
else if(avg>=50 &&avg<60) {
printf("second class");
}
else if (avg<50) {
printf("fail");
} }
```

OUTPUT

enter three subject marks80 85 81

distinction

switch statement:

switch statement is one of decision-making control-flow statements. Just like else if ladder, it is also used to make a decision among multiple choices. The switch statement is a multi-way branch statement. In the program there is a possibility to make a choice from number of options, then this structured statement is useful. switch statement has the following form:

Syntax:

switch(<exp>)

{

case <exp-val-1>: statements block-1

break;

case <exp-val-2>: statements block-2

break;

case **<exp-val-3>: statements block-3**

break;

case<exp-val-N>: statements block-N

break;

default: default statements block

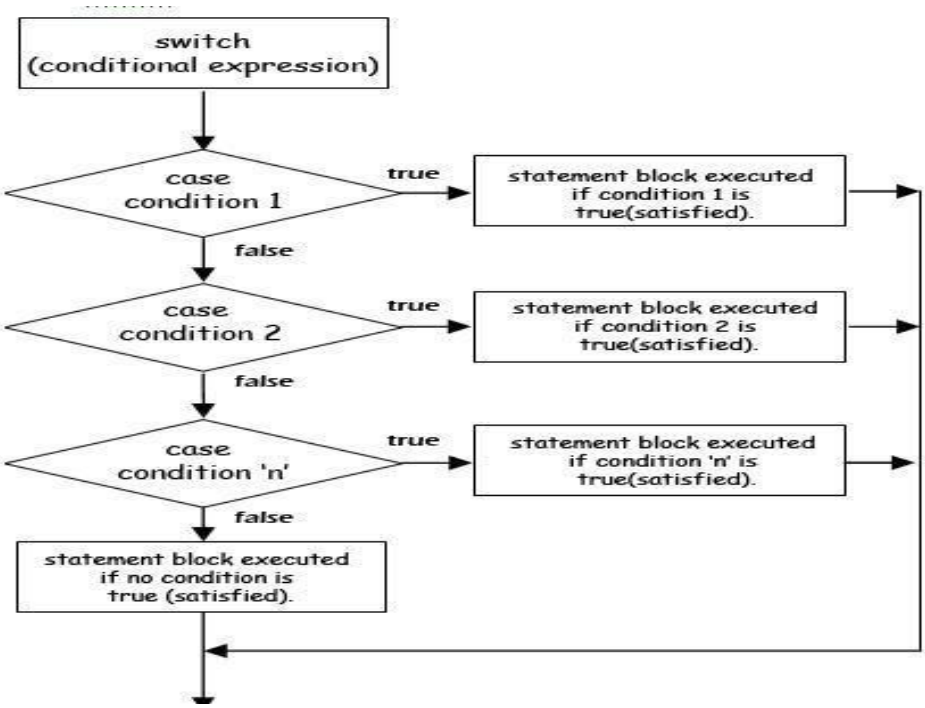
}

Next-statement;

In this syntax,

- switch, case, default and break are keywords.

- <*exp*>is any expression that should give an integer value or character value. In other words, it should never return any floating-point value. It should always be enclosed with in parentheses (and). It should also be placed after the keyword switch.
- <*exp-val-1*>, <*exp-val-2*>, <*exp-val-3*>…. <*exp-val-N*> should always be integer constants or character constants or constant expressions. In other words, variables can never be used as <*exp- val*>. There should be a space between the keyword case and <*exp-val*>. The keyword case along with its <*exp-val*> is called as a case label. <*exp-val*> should always be unique; no duplications are allowed.
- *statements block-1, statements block-2, statements block-3… statements block-N* and *default statements block* are simple statements, compound statements or null statements. It is important to note that the statements blocks along with their own case labels should be separated with a colon (:)
- The break statement at the end of each statements block is an optional one. It is recommended that break statement always be placed at the end of each statements block. With its absence, all the statements blocks below the matched case label along with statements block of matched case get executed. Usually, the result is unwanted.
- The statement block and break statement can be enclosed with in a pair of curly braces { and }.
- The default along with its statements block is an optional one. The break statement can be placed at the end of default statements block. The default statements block can be placed at anywhere in the switch statement. If they are placed at any other place other than at end, it is compulsory to include a break statement at the end of default statements block.
- *Next-statement* is a valid C statement.



Whenever, switch statement is encountered, first the value of <*exp*> gets matched with case values. If suitable match is found, the statements block related to that matched case gets executed. The break statement at the end transfers the control to the *Next-statement*.

If suitable match is not found, the default statements block gets executed and then the control gets transferred to *Next-statement*.

Example Program:

```
#include<stdio.h>
void main() {
int a,b,c,ch;
printf("\nEnter two numbers :");
scanf("%d%d",&a,&b);
printf("\nEnter the choice:");
scanf("%d",&ch);
switch(ch) {
    case 1: c=a+b;
            break;
    case 2: c=a-b;
            break;
    case 3: c=a*b;
            break;
    case 4: c=a/b;
            break;
    case 5:
            return;
    }
    printf("\nThe result is: %d",c);
    }
```

Output: enter the choice 1 Enter two numbers 4 2 The Result is: 6

****

Repetition statements (loops)

while, for, do-while statements with C Program examples

****

Repetition statements (loops):

- Iteration statements is also known as Looping statement/ Repetition statement
- A segment of the program that is executed repeatedly is called a loop
- Some portion of the program has to be specified several number of times or until a particular condition is satisfied

The three methods/ loops by which you can repeat a part of a program are,

1) while Loop

2) do ……. while loop

3) for loop

Loops generally consist of two parts :

Control expressions: One or more control expressions which control the execution of the loop,

Body: which is the statement or set of statements which is executed over and over

Any looping statement, would include the following steps:

a) Initialization of a condition variable

b) Test the control statement.

c) Executing the body of the loop depending on the condition.

d) Updating the condition variable.

1) While:

The simplest of all the looping structures in c is the while statement. The basic format of the while statement is:

Syntax:

Initialization statement;

while(condition)

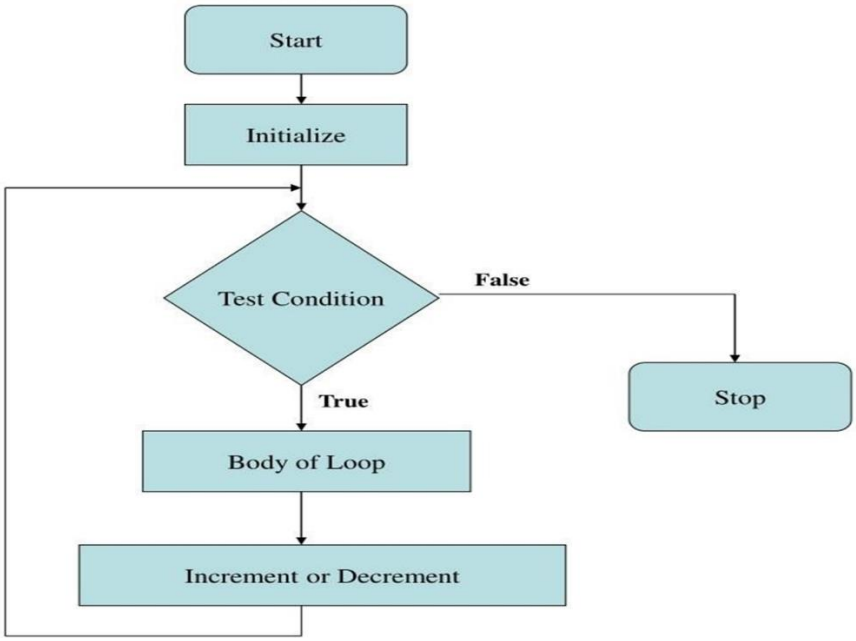
{

statements;

}

The while is an entry –controlled loop statement. The condition is evaluated and if the condition is true then the statements will be executed. After execution of the statements the condition will be evaluated and if it is true the statements will be executed once again. This process is repeated until the condition becomes false and the control is transferred out of the loop. On exit the program continues with the statement immediately after the body of the loop.

Flow chart:



Example-1: Program to print n natural numbers using while

```
#include<stdio.h>
int main () {
int i,n;
printf("enter the range\n");
scanf("%d",&n);
i=1;
while(i<=n) {
printf("%d ",i);
i=i+1;
} }
```

Output: enter the range 10

1 2 3 4 5 6 7 8 9 10

Ex-2: Summation of the series 1+2+3+4+....+10

```
#include<stdio.h>
int main() {
        int i=0,sum=0;
        while(i<=10)
        {
                sum = sum+i;
                i++;
        }
        printf("the sum value is: %d\n",sum);
        return 0; }
```

Ex-3: Summation of squares of the series 1^2+2^2+3^2+....+10^2

```
#include<stdio.h>
#include<math.h>
int main () {
        int i=1, sum=0;
        while(i<=10)
        {
                sum = sum+i*i;
                i++;
        }
printf("the sum value is: %d\n", sum);
        return 0; }
```

Ex-4: Summation of squares of the series 1^1+2^2+3^3+....+10^10

```
#include<stdio.h>
#include<math.h>
int main()
{
        long long i=1,sum=0;
        while(i<=10)
```

```
{
        sum = sum+pow(i,i);
        i++;
}
printf("the sum value is: %lld\n",sum);
return 0; }
```

Ex-5: WAP to print the summation of digits of any given number

```
#include<stdio.h>
int main() {
    int num, rem=0, sum=0;
    printf("enter any number");
    scanf("%d",&num);
    num = num < 0 ? -num : num;
    while(num!=0)
    {
        rem = num%10;
        sum = sum+rem;
        num = num/10;
    }
    printf("the summation of given number is : %d",sum);
    return 0; }
```

do-while statement:

It is one of the looping control statements. It is also called as *exit- controlled looping control statement*. i.e., it tests the condition after executing the do-while loop body. The main difference between ―while‖ and ―do-while‖ is that in ―do-while‖ statement, the loop body gets executed at least once, though the condition returns the value false for the first time, which is not possible with while statement. In ―while‖ statement, the control enters into the loop body only when the condition returns true.

Syntax:

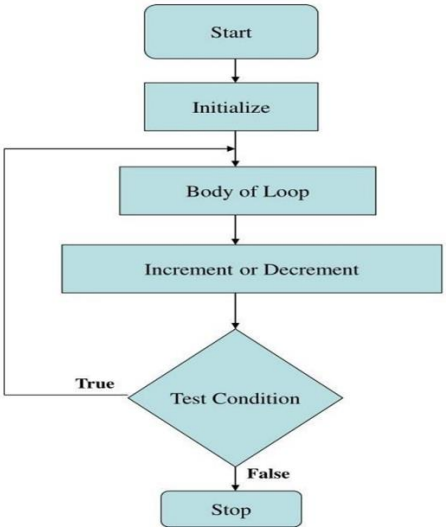
```
Initialization statement;
do {
statement(s);
```

} while(<condition>);

next statement;

In this syntax:

- **while** and **do** are the keywords. <condition> is a relational expression or a compound relational expression or any expression that returns either true or false. initialization statement, statement(s) and next_statement are valid _c_ statements. The statements within the curly braces are called as do-while loop body. The updating statement should be included with in the do-while loop body. There should be a semi-colon (;) at the end of while(<condition>).



Whenever ―do-while‖ statement is encountered, the initialization statement gets executed first. After then, the control enters into do-while loop body and all the statements in that body will be executed. When the end of the body is reached, the condition is tested again with the updated loop counter value. If the condition returns the value false, the control transfers to next statement without executing do-while loop body. Hence, it states that, the do-while loop body gets executed for the first time, though the condition returns the value false.

Ex-1: Program to print n natural numbers using using do while

```
#include<stdio.h>

int main () {

int i,n;

printf("enter the range\n");

scanf("%d",&n);

i=1;

do {

printf("%d\n",i);

i=i+1;

}while(i<=n);

}
```

Output: enter the range 8

1 2 3 4 5 6 7 8

Ex-2: WAP to print Fibonacci series for any given number using Do-While Loop

```
#include<stdio.h>
int main () {
        int num,f1=0,f2=1,f3;
        printf("Enter any number \n");
        scanf("%d",&num);
        printf("The Fibanacci series is : \n");
        printf("%d\t",f1);
        printf("%d\t",f2);
        int count=2;
        do
        {
                f3 = f1+f2;
                printf("%d\t",f3);
                f1 = f2;
                f2 = f3;
                count++;
        } while(count<num);
        return 0; }
```

OUTPUT: Enter any number 10

The Fibanacci series is

0 1 1 2 3 5 8 13 21 44

Comparisons between while and do.. while loop statement

While	do.. while
It is an entry controlled loop statement, i.e. the condition is tested before executing the statements of loop.	It is an exit controlled loop statement, i.e. it tests condition after executing the statements of the loop.
Initially, if the condition becomes false then it does not execute any statements of the loop.	Initially, it executes the statements of loop at least once even the condition is false.

for statement:

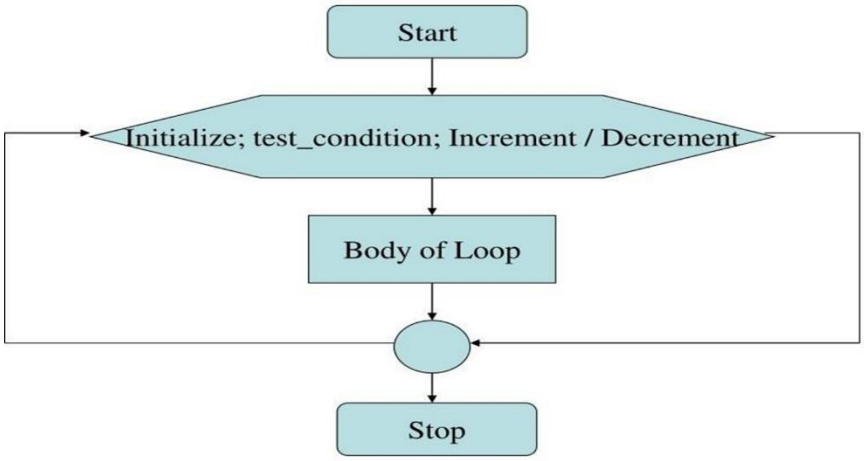
It is one of the looping control statements. It is also called as *entry-controlled looping control statement*. i.e., it tests the condition before entering into the loop body. The syntax for ‖for‖ statement is as follows:

Syntax:

```
for (exp1; exp2; exp3)
{
for-body;
}
next_statement;
```

In this syntax,

for is a keyword. exp1 is the initialization statement. If there is more than one statement, then the statements must be separated with commas. exp2 is the condition. It is a relational expression or a compound relational expression or any expression that returns either true or false. The exp3 is the updating statement. If there is more than one statement then, they must be separated with commas. exp1, exp2 and exp3 should be separated with two semi-colons. exp1, exp2, exp3, for- body and next_statement are valid —c_ statements. for-body is a simple statement or compound statement or a null statement.



- Whenever ‖for‖ statement is encountered, first exp1 gets executed. After then, exp2 is tested. If exp2 is true then the body of the loop will be executed otherwise loop will be terminated.
- When the body of the loop is executed the control is transferred back to the for statement after evaluating the last statement in the loop. Now exp3 will be evaluated and the new value is again tested. if it satisfies body of the loop is executed. This process continues till condition is false.

Ex-1: Program to print n natural numbers using for loop

```
#include<stdio.h>
void main() {
int i,n;
printf("enter the value");
scanf("%d",&n);
for(i=1;i<=n;i++) {
printf("%d ",i);
} }
```

Output: enter the value 5

1 2 3 4 5

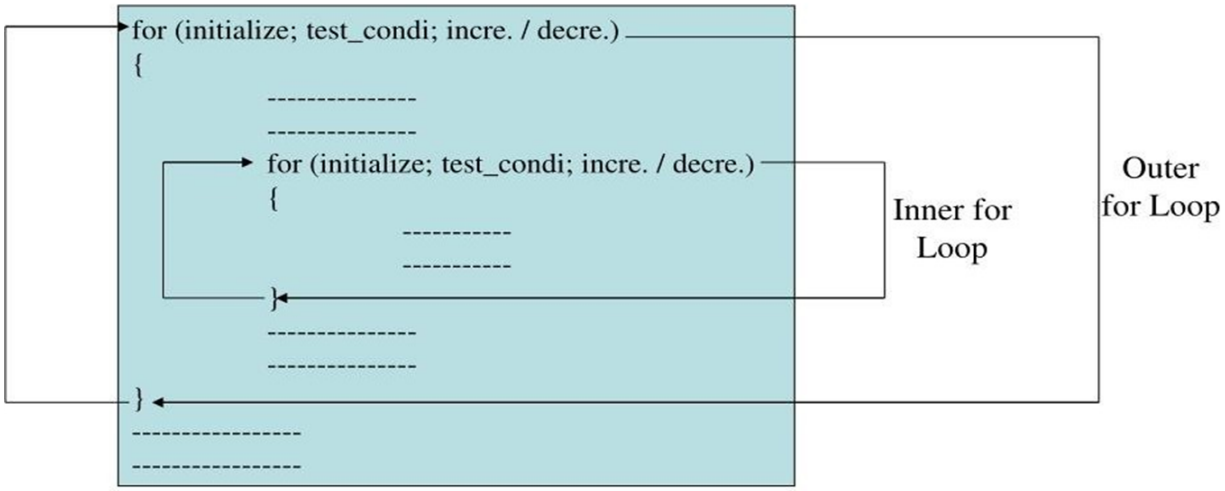
Ex-2: WAP to print multiplication table

```
#include<stdio.h>
int main() {
        int mul,limit,c,i;
        printf("Enter the multiplication number");
        scanf("%d",&mul);
        printf("Enter the limit for table");
        scanf("%d",&limit);
        for(i=1;i<=limit;i++)
        {
                c=i*mul;
                printf("%d*%d = %d\n",mul,i,c);
        }
        return 0; }
```

Nesting of for Loop

The One for statement within another for statement is called Nesting for Loop.

Syntax:



Ex-1: Print the i and j values

```
#include<stdio.h>
int main() {
        int i,j;
        for(i=1;i<=10;i++)
        {
                printf("the i value is %d\n",i);
                for(j=1;j<=10;j++)
                {
                        printf("the j value is %d\n",j);
                }
        }
        return 0; }
```

Ex-2: Multiplication Table

```
#include<stdio.h>
int main()
{
```

```
        int mul, a, b;
        for(a=1;a<=5;a++)
        {
                printf("the multiplication table for %d\n",a);
                for(b=1;b<=10;b++)
                {
                        mul = a*b;
                        printf("%d*%d = %d\n ",a,b,mul);
                }
                mul=0;
        }
        return 0;
}
```

Ex-3: Write a C program to display the prime numbers between given range.

```
#include <stdio.h>
int main() {
   int x, y, j, i, count, q;
   printf("Enter the range of numbers (x to y): ");
   scanf("%d%d", &x, &y);
   printf("Prime numbers between %d and %d are:\n", x, y);
   for (j = x; j < y; j++) {
      count = 0;
      for (i = 1; i <= j; i++) {
         if (j % i == 0) {
            count++; } }
      if (count == 2) {
         printf("%d\t", j);
      q++; }  }
   printf("\n the number of prime numbers are: %d",q);
   return 0; }
```

Pattern problems using nested for loop

Ex-1: //Square Star Pattern

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter the number of rows: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

```
Enter the number of rows: 5
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

//Right Triangle Star Pattern

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter the number of rows: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
```

```
    }
    return 0;
}
```

```
Enter the number of rows: 5
*
* *
* * *
* * * *
* * * * *
```

//Hollow Right Triangle Star Pattern

```
#include <stdio.h>
int main() {
    int i,j, n;
    printf("Enter the number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for ( j = 1; j <= i; j++) {
            if (j == 1 || j == i || i == n) {
                printf("* ");
            } else {
                printf("  ");
            }
        }
        printf("\n");
    }
    return 0; }
```

```
Enter the number of rows: 5
*
* *
*   *
*     *
* * * * *
```

//Full Pyramid of *

```
#include <stdio.h>
int main() {
    int i,j,space, n;
    printf("Enter the number of rows: ");
    scanf("%d", &n);
    for ( i = 1; i <= n; i++) {
        for ( space = 1; space <= n - i; space++) {
            printf("  ");
        }
        for ( j = 1; j <= 2 * i - 1; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

```
Enter the number of rows: 5
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

//Hollow Square Star Pattern

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter the number of rows: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n) {
```

```
            printf("* ");
        } else {
            printf("  ");
        }
    }
    printf("\n");
}
return 0; }
```

```
Enter the number of rows: 5
* * * * *
*       *
*       *
*       *
* * * * *
```

//Hollow Square Pattern with Diagonal

```
#include <stdio.h>
int main() {
    int i, j, n;
    printf("Enter the number of rows: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n || i == j || j == (n - i + 1)) {
                printf("* ");
            } else {
                printf("  ");
            }
        }
        printf("\n");
    }
    return 0; }
```

```
Enter the number of rows: 5
* * * * *
* *   * *
*   *   *
* *   * *
* * * * *
```

****

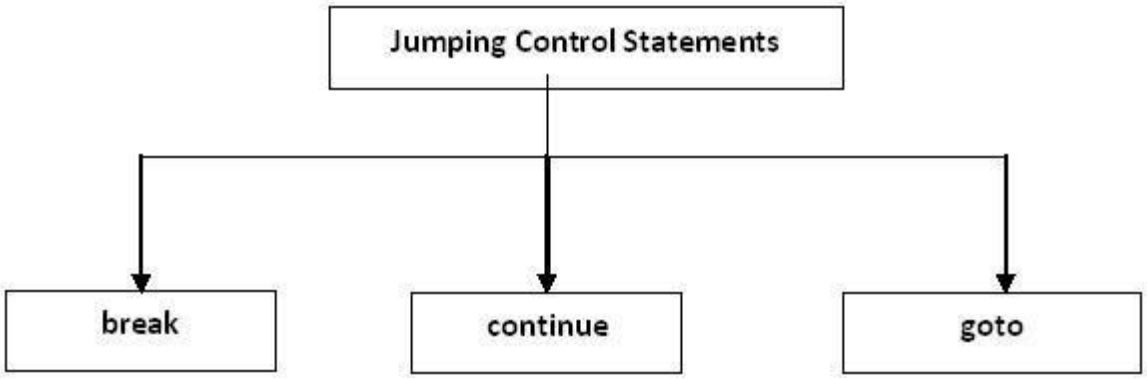
Statements related to looping

break, continue, goto, Simple C Program examples.

****

Jump (or) Loop Control (or) Unconditional Statements

Jumping control-flow statements are the control-flow statements that transfer the control to the specified location or out of the loop or to the beginning of the loop. There are 3 jumping control statements:



These statements transfer control to another part of the program. When we want to break any loop condition or to continue any loop with skipping any values and to directly jump to termination of the program, we use these statements. There are four types of jumps statements.

- break statement
- continue statement
- goto statement

1. break statement:

The ―break‖ statement is used within the looping control statements, switch statement and nested loops. When it is used with the for, while or do-while statements, the control comes out of the corresponding loop and continues with the next statement. When it is used in the nested loop or switch statement, the control comes out of that loop / switch statement within which it is used. But, it does not come out of the complete nesting.

Syntax for the ―break‖ statement is:

break;

In this syntax, break is the keyword.

The following representation shows the transfer of control when break statement is used:

```
Any loop
{
statement_1;
statement_2;
break;
```

}
next_statement;

Program for break statement:

```
#include<stdio.h>
int main () {
int i;
for(i=1; i<=10; i++)
{
if(i==6)
break;
printf("%d",i);
} }
```

Output: 12345

2. Continue statement

A continue statement is used within loops to end the execution of the current iteration and proceed to the next iteration. It provides a way of skipping the remaining statements in that iteration after the continue statement. It is important to note that a continue statement should be used only in loop constructs and not in selective control statements.

Syntax for continue statement is:

continue;

- where continue is the keyword.
- The following representation shows the transfer of control when continue statement is used:

Any loop

{

statement_1;

statement_2;

continue;

} next_statement;

Ex: Program for continue statement:

```
#include<stdio.h>
int main() {
int i, sum=0, n;
for(i=1; i<=10; i++) {
        printf("enter any no:");
```

```
        scanf("%d",&n);
        if(n<0)
                continue;
        else
                sum=sum+n;
printf("%d\n",sum); }
}
```

Output: enter any no:8 18

3. goto statement

The goto statement transfers the control to the specified location unconditionally. There are certain situations where goto statement makes the program simpler. For example, if a deeply nested loop is to be exited earlier, goto may be used for breaking more than one loop at a time. In this case, a break statement will not serve the purpose because it only exits a single loop.

Syntax for goto statement is:

label:

{

statement_1;

statement_2;

}

goto label;

- In this syntax, goto is the keyword and label is any valid identifier and should be ended with a colon (:).
- The identifier following goto is a statement label and need not be declared. The name of the statement or label can also be used as a variable name in the same program if it is declared appropriately. The compiler identifies the name as a label if it appears in a goto statement and as a variable if it appears in an expression.

If the block of statements that has label appears before the goto statement, then the control has to move to backward and that goto is called as backward goto. If the block of statements that has label appears after the goto statement, then the control has to move to forward and that goto is called as forward goto.

Program for goto statement:

```
#include<stdio.h>
void main() {
printf("www.");
goto x;
```

```
y:
        printf("expert");
        goto z;
        x:
        printf("c programming");
        goto y;
        z: printf(".com"); }
```

Output: www.c programming expert.com